

Codes et réductions — guide d'intégration mobile

Un seul champ à l'achat, trois usages derrière. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.
Référence : issue #247. Remplace la section « code » du [guide parrainage](#).

1. La règle, en une phrase

L'acheteur saisit UN code. Le serveur décide de son effet : une réduction sur le prix, une commission au propriétaire du code, ou les deux.

Il n'y a **pas** de champ séparé « code promo » et « code parrain ». L'utilisateur ne sait pas — et n'a pas à savoir — de quelle nature est le code qu'on lui a donné.
Un code ne porte pas un « type » mais un ensemble d'**effets**, qu'il cumule :

effet	conséquence
REDUCTION	baisse le prix de l'abonnement
COMMISSION	verse une part du prix au propriétaire du code
ABONNEMENT_OFFERT	ouvre l'abonnement sans paiement

Un code de parrainage porte `COMMISSION`. Ce qu'on appelait « ambassadeur », c'est `REDUCTION` + `COMMISSION`. Le client n'a pas à connaître ces noms — il lit ce que la réponse annonce.
Un code invalide ne fait jamais échouer la souscription. Elle réussit sans remise ni commission.
Authentification. Jeton JWT requis. **Scope pays** habituel.

2. Vérifier un code avant de payer — `POST /codes/valider`

À appeler quand l'utilisateur finit de saisir, pour afficher le prix barré.

```
{ "code": "amba247", "plan_uuid": "574612e9-...", "pays": "benin" }
```

La casse et les espaces sont normalisés : `amba247` et `AMBA247` désignent le même code.

Code accepté — 201

```
{
  "valide": true,
  "code": { "uuid": "9172b4e6-...", "code": "AMBA2", "origine": "ADMIN", "effets": ["REDUCTION", "COMMISSION"] },
  "effets": {
    "remise": { "type": "POURCENTAGE", "valeur": 15, "montant_remise": 300, "prix_initial": 2000, "prix_final": 1700 },
    "commission_pour": 26756
  }
}
```

Tout ce qui compte pour l'écran est dans **effets** :

clé présente	à afficher
remise	prix barré → <code>prix_final</code>
abonnement_offert	« Cet abonnement vous est offert » — aucun paiement
commission_pour	rien ; c'est une affaire entre le serveur et le parrain

N'appliquez pas le pourcentage vous-même : arrondi et plafonnement sont faits côté serveur, et un calcul local finirait par diverger d'un franc.
Un code de parrainage est **valide avec un objet effets presque vide** — il ne réduit rien. Ne traitez pas ça comme une erreur.

Le cas particulier : `abonnement_offert`

```
{ "valide": true, "effets": { "abonnement_offert": { "duree_jours": 90 } } }
```

Le tunnel de paiement doit être court-circuité. L'abonnement sera actif dès la souscription, sans encaissement. Afficher un montant à régler serait un contresens ; le bouton doit dire « Activer », pas « Payer ».

`duree_jours` peut être absent : la durée du plan s'applique alors.

Code refusé — 201 aussi

```
{ "valide": false, "motif": "INTROUVABLE" }
```

Ce n'est pas une erreur HTTP. Un code refusé est une réponse normale : testez `valide`, pas le statut.

motif	message suggéré
INTROUVABLE	Ce code n'existe pas.
INACTIF	Ce code n'est plus valable.
EXPIRE	Ce code a expiré.
PAS_ENCORE_VALIDE	Ce code n'est pas encore utilisable.
QUOTA_TOTAL_ATTEINT	Ce code a atteint son nombre d'utilisations.
DEJA_UTILISE	Vous avez déjà utilisé ce code.
PLAN_NON_ELIGIBLE	Ce code ne s'applique pas à cette formule.
AUTO_UTILISATION	Vous ne pouvez pas utiliser votre propre code.

`QUOTA_TOTAL_ATTEINT` et `DEJA_UTILISE` se ressemblent mais n'appellent pas le même message : le premier veut dire « trop tard », le second « déjà servi ».

3. Souscrire avec un code — `POST /abonnements/souscrire`

```
{ "plan_uuid": "574612e9-...", "code": "AMBA247", "pays": "benin" }
```

Le champ s'appelle `code`. `code_parrainage` reste accepté comme alias pour les versions déjà déployées, mais n'accepte que les codes de parrainage dans l'esprit ; utilisez `code`.

Réponse — 201

```
{
  "uuid": "abo-...", "statut": "EN_ATTENTE", "offert": false,
  "code_id": 52938, "montant_remise": 300,
  "parrain_id": 26756, "montant_paye": 0
}
```

Avec un code `ABONNEMENT_OFFERT` :

```
{
  "uuid": "abo-...", "statut": "ACTIF", "offert": true,
  "date_debut": "2026-09-06T...", "date_fin": "2026-12-05T...",
  "code_id": 52939, "montant_remise": 0, "parrain_id": null, "montant_paye": 0
}
```

champ	interprétation
<code>code_id</code> non nul	le code a été retenu et sa place consommée
<code>code_id: null</code>	aucun code retenu — absent, invalide, ou place prise entre-temps
<code>montant_remise</code>	remise obtenue, en devise du plan
<code>offert: true</code>	rien à payer : <code>statut</code> est déjà ACTIF , le droit est ouvert
<code>parrain_id</code>	bénéficiaire de la commission, s'il y en a un

⚠ **offert: true change le parcours.** N'envoyez pas l'utilisateur vers le paiement : l'abonnement est actif, montrez-lui sa date de fin. Un abonnement offert ne verse jamais de commission — rien n'a été encaissé.

⚠ **La place peut avoir été prise entre l'aperçu et l'achat.** Un code limité à 100 personnes peut atteindre son plafond pendant que l'utilisateur remplit le formulaire. Le serveur souscrit alors **sans le code** plutôt que d'échouer le paiement. Comparez donc `montant_remise` à ce que l'aperçu annonçait avant d'afficher le prix final — c'est le seul moment où l'écart se voit.

`montant_paye` vaut `0` tant que l'abonnement n'est pas encaissé ; le prix à régler est `prix du plan - montant_remise` .

4. Ce que l'écran doit faire

- Un seul champ, bien libellé.** « Code promo ou parrainage » plutôt que « Code promo » : l'utilisateur ne sait pas ce qu'il a en main.
- Valider à la saisie, pas au paiement.** Un code refusé découvert après le tunnel de paiement est une mauvaise surprise ; `POST /codes/valider` sert exactement à l'éviter.
- Afficher le prix barré.** `prix_initial` → `prix_final` rend la remise tangible. Sans cela, l'utilisateur doute que son code ait servi.
- Revérifier après la souscription.** Si `montant_remise` est inférieur à l'aperçu, dites-le : « Ce code n'était plus disponible, votre abonnement se poursuit au prix normal. »
- Ne pas bloquer sur un code refusé.** Le champ est facultatif ; laissez toujours la souscription possible.

5. Exemple Flutter

```
class CodesApi {
  CodesApi(this._client, {required this.baseUrl, required this.token, required this.pays});
  final http.Client _client;
  final String baseUrl, token, pays;

  Future<ResultatCode> valider(String code, String planUuid) async {
    final r = await _client.post(
      Uri.parse('$baseUrl/codes/valider'),
      headers: {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'},
      body: jsonEncode({'code': code.trim(), 'plan_uuid': planUuid, 'pays': pays}),
    );
    // Un code refusé revient en 201 : c'est `valide` qui tranche, pas le statut.
    return ResultatCode.fromJson(jsonDecode(r.body));
  }
}

class ResultatCode {
  const ResultatCode({required this.valide, this.motif, this.remise});
  final bool valide;
  final String? motif;
  /// `null` sur un code de parrainage : il ne réduit rien, il désigne un
  /// bénéficiaire de commission. Ce n'est pas une erreur.
  final Remise? remise;

  String get message => switch (motif) {
    'INTROUVABLE' => 'Ce code n'existe pas.',
    'INACTIF' => 'Ce code n'est plus valable.',
    'EXPIRE' => 'Ce code a expiré.',
    'PAS_ENCORE_VALIDE' => 'Ce code n'est pas encore utilisable.',
    'QUOTA_TOTAL_ATTEINT' => 'Ce code a atteint son nombre d'utilisations.',
    'DEJA_UTILISE' => 'Vous avez déjà utilisé ce code.',
    'PLAN_NON_ELIGIBLE' => 'Ce code ne s'applique pas à cette formule.',
    'AUTO_UTILISATION' => 'Vous ne pouvez pas utiliser votre propre code.',
    _ => 'Code non pris en compte.',
  };
}
```

Souscrire, puis vérifier ce qui a réellement été appliqué

```
final apercu = await codes.valider(champCode.text, plan.uuid);
final abonnement = await abonnements.souscrire(plan.uuid, code: champCode.text);

// La place a pu partir pendant que l'utilisateur remplissait le formulaire.
final attendu = apercu.remise?.montantRemise ?? 0;
if (attendu > 0 && abonnement.montantRemise < attendu) {
  afficherInfo('Ce code n'était plus disponible. Votre abonnement se poursuit au prix normal.');
```

```
}

final aRegler = plan.prix - abonnement.montantRemise;
```

6. Rappels

Un seul champ pour tous les codes. Réduction, ambassadeur, parrainage : le registre tranche, pas l'application.

Un code refusé revient en 201. Testez `valide`, jamais le statut HTTP.

Un code valide sans `remise` est normal — c'est un code de parrainage.

`offert: true` court-circuite le paiement. Vérifiez ce champ avant d'ouvrir le tunnel d'encaissement.

Ne recalculiez pas la remise. Arrondi et plafonnement sont faits côté serveur ; une remise de 100 % rend l'abonnement gratuit, jamais un prix négatif.

Vérifiez `montant_remise` après la souscription. L'aperçu peut dater : c'est là que se voit une place prise entre-temps.

La casse et les espaces sont ignorés. Inutile de forcer les majuscules à la saisie — mais ça reste plus lisible à l'écran.